Integration report — what I changed in the second implementation, and why

Author: Paridhi Mittal · **Branch:** `integrate-friend-impl` · **Period:** 15–16 October 2026 **Baseline:** my teammate's implementation at commit `957224c` (merged in `bed240b`)

Two of us built independent implementations of the assignment. I compared them, decided to build on my teammate's (it had the complete multi-agent graph, harness and diagnosis), and carried over what my own implementation did better: the agent contracts, the data rules and the request validation. This report records every change I made to the second implementation, the reason for each, and the evidence behind it. AI assistance used in this work is disclosed in `AI_USE.md` (§§5–6); the decisions recorded here were mine.

---

1. Summary

	Before (`957224c`)	After (`HEAD`)
Canonical population	4,578 listings, months in UTC, drop-on-defect	4,975 sale dwellings, months in IST; 4,574 with a usable price, the rest flagged, not dropped
Output contract	per-month `n_active`, `median_psf`	the design note's five monthly fields plus extensions (`contracts/CONTRACT.md` v2.1.0-draft)
Contracts	none	a global contract and one per agent, 12 files in `contracts/`
Relations	11	12 (added M4, radius halving)
Base inputs	36 locality inputs	72: 36 locality + 36 radius
Statistical unit	the run (per-run Wilson CI)	the base input (a base input is violated if any seed is)
Geography widening	dropped the locality filter (whole metro)	keeps it, and provably preserves set inclusion
Coverage loop	one threshold, levers price → BHK → geography	30-listing coverage, ACCEPT / RETRY / LIMITED, two retries, geography → BHK → price band
Critique	APPROVED / REJECTED	adds coherence checks and NEEDS_REVIEW
Request validation	none	`src/validation.py`, ported from my implementation
LLM sweep	pre-v2 only	re-run under v2 on a Kaggle T4, replayable offline
Tests	84 functions (101 cases)	132 functions (154 cases), all passing
Reproducibility checks	headline table only	headline tables for both sweeps, `verify_claims.py`, offline LLM replay, logged check runs
Report	7,431 words, pre-v2 results	rewritten for v2; 10 A4 pages, as the brief asks

Faults found and fixed in the inherited code: geography widening that broke set inclusion (found by M3), an LLM review batch cut before duplicates were removed (found by I3), a merge that returned both rows of a repeated listing ID, a "model digest" that was a hash of the model's name, two broken scripts, and figures in the report that no script could reproduce (§4).

---

2. How I organised the work

I split the integration into phases and checked each before starting the next (`NOTEBOOK.md` Stages 16–24):

Phase	What	Main commits
0	Merge the second implementation into an integration branch; move my first implementation to `legacy/`; make the snapshot checksum survive Windows	`bed240b`, `b8b33c6`, `bde303c`
1	Decide the contract: which rules from each implementation win	`contracts/CONTRACT.md` §11
2	Write the contract suite, one file per agent	`e6e69f2`, `109bc26`
3	Port my data rules and request validation	`4d6d51f`, `ba6a039`, `02862e5`
4	Fix the harness and the agents; add radius inputs and M4; re-run the sweeps	`02cc5ea`, `c78a270`, `1d7670e`, `ba15231`, `8148658`

Phase	What	Main commits
5	Update the report, design note, diagnosis and AI-use record	`f28ef59`, `65db1ec`
6	Align with the team's design note (contract v2.1) and re-run	`f4752f8`, `9fca01f`, `5b9271e`
7	Re-run the LLM sweep on Kaggle, find and fix the I3 fault, verify by replay	`c8e6e85`, `90ceaac`, `daa4768`
8	Cut the report to 10–12 pages; logs, issue index, deliverables check	`6bb3fd2`, `28fe9cd`, `c1b3043`

Decisions I made along the way:

- **Build on the second implementation** rather than refine mine: it had the full agent graph, the MR harness and a careful diagnosis; mine had better contracts, data rules and validation, which were easier to port than to rebuild the rest.
- **Report both price bounds** (decision 7 in `contracts/CONTRACT.md` §11): `MIN_PRICE` and `MAX_PRICE` medians are both kept, so a range listing would show; the representative price became the midpoint when we adopted the design note.
- **Fix the M3 fault rather than only report it**, and keep the failing sweep as evidence (`results/v2_before_m3fix/`).
- **Adopt the design note, but keep ten argued deviations** (`CONTRACT.md` §13), each with its evidence.
- **Re-run the LLM sweep after the I3 fix** instead of reporting the fault as open, keeping the first run as evidence (`results/v2_llm_before_reviewfix/`).
- **Choose the LLM sweep's top three** (M1raw, M1, D2) for the report's root-cause analysis, since it is the complete system.

---

3. Changes by area

3.1 Repository and platform

Change	Why
`.gitattributes`: `-text` for `data/**`, `results/**`, `tests/golden/**`, `legacy/data/**`, `audit/**`	On Windows, git's line-ending conversion rewrote the frozen snapshot and the snapshot's SHA-256 check failed. Pinned files must be byte-identical on every platform.
UTF-8 I/O in every script; ASCII table headers	Scripts crashed reading "₹" under the Windows code page, and a "≠" header crashed redirected output.
My first implementation moved to `legacy/` (`b8b33c6`)	Keeps its contracts, audits and test logs available without two codebases competing.
`.gitignore`: quick-run results ignored	Scratch sweeps (`results/quick/`) do not belong in the history.

3.2 Data loading — `src/loading.py`

Change	Why
Population = sale dwellings with a parseable `POSTING_DATE`: **4,975** listings	Rent and PG rows carry rents; land has no built-up area. Mixing them makes ₹/sqft meaningless.
Months derived in **Asia/Kolkata**, not UTC	18 listings fell into the wrong month in UTC.
Price problems **flag** a row (`price_valid`) instead of dropping it	The design note counts every listing (`listing_count`) but computes prices only from usable ones (`price_valid_count`); "no usable price" must be distinguishable from "no listing".
"Price on Request" listings not price-valid; `MAX_PRICE` validity and ordering checked	Ported from my implementation: those bounds are placeholders, not asking prices.
`listing_price_inr` = (MIN + MAX) / 2	The design note's representative asking price.
`locality_id` parsed from the raw data	The design note filters by locality ID as well as name.
`listing_id` = `PROP_ID#row`	One `PROP_ID` appears twice; the old merge returned both rows whenever either matched.

Change	Why
`RejectLog.flag`; `data/snapshot/canonical_price_validity.csv`	Every flagged row is recorded with its reason.
`freeze_canonical` writes fixed line endings; canonical SHA pinned	The derived snapshot is reproducible byte for byte.

3.3 Output contract — `src/contract.py`

Change	Why
`CONTRACT_VERSION = "2.1.0-draft"`; `MonthStat` gains `listing_count`, `price_valid_count`, `median_listing_price_inr`, `coverage_status`, min/max-price medians, `is_partial_month`	The design note's five monthly fields; every relation asserts on these.
`coverage_status()`: ADEQUATE (≥ 30 usable listings) / LIMITED / NO_DATA_IN_SNAPSHOT	The note's coverage rule, distinct from the headline's sufficiency threshold (n ≥ 50), which stays an extension (deviation D-3).
`TrendReport` series methods for each field; `n_price_valid_total`	Relations and the dashboard read named series rather than recomputing.
`PARTIAL_MONTH = "2023-08"`	The scrape ends on 26 August; the summary must say so.

3.4 Request validation — `src/validation.py` (new, ported)

`validate_request` and `SourceCatalog` check a request against what the snapshot actually contains — localities and their IDs, cities, BHK, price bands, window, radius centre — and return coded errors (`unsupported_city`, `invalid_geo_center`, `query_mode_mismatch`, …). The second implementation accepted any request and failed late or silently; validation was the part of my implementation I most wanted to keep. `GEO_RADIUS` is accepted as the design note's name for `RADIUS`.

3.5 Statistics — `src/trend.py`

Change	Why
City filter; localities match by name **or** ID	The design note's filters.
Price band by **range overlap** (`MAX_PRICE ≥ min` and `MIN_PRICE ≤ max`)	A listing whose range touches the band belongs in it.
Medians and rankings from price-valid rows only	Flagged rows count but carry no price.
`adequate_n` parameter	The coverage threshold is a contract value, not a literal.

3.6 Agents — `src/agents/nodes.py`, `src/agents/state.py`

Change	Why
Geography widening keeps the locality constraint (`c78a270`)	It used to drop the locality filter, so every narrow query became a whole-metro query — a different question — and relation M3 could not fail.
Widening preserves set inclusion (`8148658`): each selected locality adds its own nearest neighbours, skipping localities already selected, until depth roughly doubles	The first version widened around the selection's centre, so a subset and its superset widened differently: **M3 failed on 9/36 inputs**. With the new rule widen(A) ⊆ widen(B) whenever A ⊆ B (argued in the code, tested on all 36 M3 pairs and 40 random subsets). M3 0/36, M2 2/72 → 0/72.
Coverage: verdicts ACCEPT / RETRY / LIMITED; at most two retries; lever order geography → BHK → price band; stops when the snapshot cannot cover more months	The design note's control step. LIMITED is reported rather than answering a broader question.
Rejected listings marked `price_valid = False` instead of dropped	`listing_count` stays stable through cycle 2; only the price statistics change.
`n_rejected_outliers` counts the current pass, not a running sum (renamed from `n_rejected_strict`)	The running sum double-counted rejections whenever the critique ran more than once.
The Critique inspects `analysed_listings`, the listings the report was computed from (declared in the state so LangGraph keeps it)	It was inspecting a different set of listings from the one the statistics came from.
`n_no_coordinate` counts only rows that match the criteria	It counted unlocated rows outside the query.

Change	Why
Critique: coherence invariants; **NEEDS_REVIEW** when a new problem remains after the one allowed recomputation	The design note's critique outcome; the cap of one recomputation is kept because a second was measured to cause violations (2/16 vs 0/16).
Aggregator: `review_status`, `coverage_verdict`, `contract_version`, `snapshot_sha256` and `narrative_source` in provenance; narrative states coverage, the partial month and NEEDS_REVIEW	Every report says how it was produced.
LLM review: duplicates removed before the batch is cut (`90ceaac`)	The review took the 12 highest-₹/sqft rows and only then removed duplicates; duplicating the data showed the model 8 distinct listings instead of 10. **I3 failed on 8/72 inputs** in the LLM sweep; after the fix, 0/72. This line was in the original code and also caused the residual the earlier report had left open.
Narrative: the faithfulness guard accepts figures in the area it was given; no model summary when there is no price change (`45c8d13`, `2edd7bf`)	Radius summaries were rejected for repeating "within 6 km of 17.440, 78.348"; with no change, the 3B model copied the facts block instead of writing prose (58/72 rejected).

3.7 Local model client — `src/llm.py`

Change	Why
Record the **weights digest** Ollama reports (`357c53fb…`)	The previous "model digest" was a hash of the model *name*, which would not detect a re-pulled model with different weights.
One real call before a sweep; stop after 10 backend errors in a row (`BackendUnavailable`)	On Kaggle a run lost its model server and every call fell back silently — it would have finished as a deterministic sweep labelled LLM.
Replies saved every 50, written atomically	An interrupted sweep keeps its replies, and a restart replays them.

3.8 Metamorphic harness — `src/mr/`

Change	Why
36 radius base inputs and relation **M4** (halve the radius)	The radius algorithm is half of the genuine dependency and was never exercised by a relation.
Base input as the statistical unit; per-run rate and seed disagreements also reported	Three seeds of one input are not independent samples: 0/108 runs gave an upper bound of 0.034 where the 36 inputs it rests on support only 0.096.
Every run records `source_rejected` / `follow_rejected`	The real mechanism that can change the analysed population is cycle 2's outlier filter; the earlier "no listing crosses a plausibility bound" precondition was vacuous because the loader applies no such bounds.
Transforms keep MIN, MAX and midpoint consistent and recompute ₹/sqft	Otherwise M1 and D2 would test an inconsistent listing.
Relations compare `listing_count`, `price_valid_count` and `coverage_status`; I3 doubles both counts; D1 compares statuses too	They assert on the new contract's fields.
M3 uses an explicit check instead of `assert`	An `assert` disappears under `python -O`.

3.9 Scripts

Script	Change	Why
`run_harness.py`	radius inputs on real data; UTF-8, fixed line endings	v2 sweeps
`headline_table.py`	`--v2`, `--tag`, `--check` against the report	the brief's one-command reproduction, for both sweeps
`verify_claims.py`	re-derives every figure for the v2 population and 12 relations; scans the dashboard for stale names	quoted numbers cannot drift from the data
`threshold_sensitivity.py` (new)	regenerates the threshold-sensitivity table	the table had been committed without a script; the new one reproduces it exactly
`regenerate_golden.py` (new)	regenerates golden vectors only with a recorded reason	no silent golden updates
`diagnose.py`, `make_synthetic.py`	repaired; write all canonical columns	both were broken before my changes

Script	Change	Why
`replay_llm_sweep.py` (new)	replays the LLM sweep from the reply cache, no model allowed	graders can check the LLM results without a GPU
`llm_fallback_breakdown.py` (new)	per-agent fallback counts and reasons	found the narrative cause of a 0.2 fallback rate
`run_checks.py` (new)	every check, logged to `audit/test_logs/`	a record of each check run
`build_report_pdf.py` (new)	renders the report and counts pages	the brief's 10–12 pages, measured

3.10 Dashboard — `app.py`, `src/viz.py`

City selection, an exact-locality / `GEO_RADIUS` switch (localities ignored in radius mode, with a note), request validation with its error messages, listing and price-valid counts with coverage status per month, a "months ADEQUATE" metric, the Relations tab counting violating base inputs and runs, and axis labels that state each series' denominator. Covered by `tests/test_app.py`, which runs the dashboard headless in both modes.

3.11 Tests — 84 → 132 functions

New: `test_validation.py` (request validation), `test_app.py` (dashboard). Extended: data rules and price validity (`test_loading`), the new contract fields (`test_contract`, `test_trend`), set inclusion of geography widening, NEEDS_REVIEW, stable counts through cycle 2 and the coverage verdict (`test_agents`), planted faults the harness must detect and the base-input interval (`test_harness`), the safeguards and the I3 and narrative fixes (`test_llm`). The golden vectors were regenerated once per intentional change with the reason recorded in `_corrections`. The expected failure that pinned the old I3 residual now passes and is a regular test.

3.12 Documents

- **`contracts/`** (new): the global contract and one per agent — Data, Criteria, Retrieval, Coverage, Analysis, Critique, Aggregator, LLM, Testing, Golden — in the format of my first implementation's contracts, with the ten design-note deviations and their evidence in `CONTRACT.md` §13.
- **`REPORT.md`** / **`REPORT.pdf`**: rewritten around the v2 sweeps and cut to 10 pages; the pre-v2 diagnoses moved, unchanged, to `docs/DIAGNOSIS.md`.
- **`docs/DIAGNOSIS.md`**: Findings 10 (M3) and 11 (I3 under the model).
- **`docs/ISSUE_LOG.md`**: every issue found, with what caught it and the fix.
- **`NOTEBOOK.md`** Stages 16–24, **`AI_USE.md`** §§5–6, **`DESIGN.md`**, **`README.md`**, **`data/SOURCE.md`**.

---

4. Faults I found in the inherited code

Fault	How I found it	Fix
Geography widening dropped the locality filter, so M3 could never fail	counting how often widening fired: every run	widening keeps a locality constraint
Widening around the selection's centre broke set inclusion	**M3 9/36** once it could fail	per-locality widening, provably inclusion-preserving
LLM review cut its batch before removing duplicates	**I3 8/72** in the v2 LLM sweep; captured prompts (10 vs 8 listings)	de-duplicate, then cut; re-run: I3 0/72
Merge returned both rows of a repeated `PROP_ID`	merge audit	unique `listing_id`
"Model digest" was a hash of the model name	trying to verify the model record	weights digest recorded
Critique inspected listings other than those the report was computed from; rejections double-counted across rounds	checking the accounting against the trace	`analysed_listings` declared and inspected; per-pass count
Threshold-sensitivity table had no script; "+6.24% radius slice" not reproducible	regenerating every quoted number	script added; the figure withdrawn
`diagnose.py` and the synthetic path no longer ran	running them	repaired

---

5. Results before and after

The earlier sweeps used 4,578 listings, 11 relations, 36 inputs and per-run intervals, so the rates are not directly comparable; what matters is which relations fail and why.

Relation	Before: deterministic / LLM (per run)	After: deterministic / LLM (per base input)
I1–I5	0 / I3 0.176, others 0	0 / 0 (I3 fixed)
M1	0.139 / 0.333	0.194 / 0.375
M2	0 / 0.111	0 / 0
M3	0 (could not fail) / 0	0 after the fix (0.250 before) / 0
M4	—	0 / 0
M1raw	0.861 / 0.806	0.718 / 0.746
D1	0 / 0.667	0 / 0.069
D2	0 / 0.593	0 / 0.264

Every remaining violation is attributed in `REPORT.md` §8: M1raw is an ill-conceived relation kept as a counter-example; M1 combines a data artifact of the outlier filter with the model judging shifted prices; D1 and D2 change what the model judge is shown.

---

6. What I kept from the second implementation

The LangGraph architecture with its two real cycles, the four parallel retrieval agents with stable hashing, the prompt-hygiene rules (listings by position, total ordering, content addressing, counts withheld, canonical units), the synthetic control snapshot, the controlled-experiment approach to diagnosis, and the reply cache. These were sound; my changes made them hold under the new contract and fixed the places where they did not.

---

7. How to check this

```
git diff 957224c HEAD --stat                       # every file changed
python -m pytest                                   # 154 tests
python scripts/run_checks.py --replay              # all checks, logged
python scripts/headline_table.py --v2 --check      # deterministic table vs the report
python scripts/headline_table.py --v2 --tag v2_llm --check
python scripts/replay_llm_sweep.py                 # the LLM sweep, offline
```